AWS VPC Peering Project using Terraform

Project Objective

Build a VPC Peering architecture in AWS using Terraform:

- VPC-1 (10.0.0.0/16)
- VPC-2 (192.168.0.0/16)
- Public Subnet in each VPC
- Internet Gateway for each VPC
- Route Tables and Associations
- Security Groups
- EC2 Instance in each VPC
- VPC Peering Connection
- Route propagation through Peering
- Connectivity verification using Private IP addresses

Project Structure

```
vpc-peering-project/
├── provider.tf
├── variables.tf
├── terraform.tfvars
├── versions.tf
├── vpc.tf
├── route.tf
├── security.tf
├── peering.tf
├── ec2.tf
├── outputs.tf
└── README.md
```

Step 1: Configure AWS CLI

```
aws configure
```

Verify:

```
aws sts get-caller-identity
```

Step 2: Provider Configuration

provider.tf

```
provider "aws" {
  region = var.region
}
```

Step 3: Create Variables

variables.tf

```
variable "region" {}

variable "vpc1_cidr" {}

variable "vpc2_cidr" {}

variable "instance_type" {}

variable "key_name" {}
```

Step 4: Variable Values

terraform.tfvars

```
region        = "us-east-1"

vpc1_cidr     = "10.0.0.0/16"
vpc2_cidr     = "192.168.0.0/16"

instance_type = "t3.micro"

key_name      = "VPC-Prod-Terraform"
```

Note:

- Use the AWS Key Pair Name only.
- Do NOT use `.pem` extension.

Step 5: Create VPCs and Subnets

vpc.tf

```
resource "aws_vpc" "vpc1" {
  cidr_block           = var.vpc1_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "VPC-1"
  }
}

resource "aws_vpc" "vpc2" {
  cidr_block           = var.vpc2_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "VPC-2"
  }
}

resource "aws_subnet" "subnet1" {
  vpc_id                  = aws_vpc.vpc1.id
  cidr_block              = "10.0.1.0/24"
  map_public_ip_on_launch = true

  tags = {
    Name = "Subnet-1"
  }
}

resource "aws_subnet" "subnet2" {
  vpc_id                  = aws_vpc.vpc2.id
  cidr_block              = "192.168.1.0/24"
  map_public_ip_on_launch = true

  tags = {
    Name = "Subnet-2"
  }
}

resource "aws_internet_gateway" "igw1" {
```

```
  vpc_id = aws_vpc.vpc1.id

  tags = {
    Name = "IGW-VPC1"
  }
}

resource "aws_internet_gateway" "igw2" {
  vpc_id = aws_vpc.vpc2.id

  tags = {
    Name = "IGW-VPC2"
  }
}
```

Step 6: Create Route Tables

route.tf

```
resource "aws_route_table" "rt1" {
  vpc_id = aws_vpc.vpc1.id
}

resource "aws_route_table" "rt2" {
  vpc_id = aws_vpc.vpc2.id
}
```

Internet Routes:

```
resource "aws_route" "internet1" {
  route_table_id         = aws_route_table.rt1.id
  destination_cidr_block = "0.0.0.0/0"
  gateway_id             = aws_internet_gateway.igw1.id
}

resource "aws_route" "internet2" {
  route_table_id         = aws_route_table.rt2.id
  destination_cidr_block = "0.0.0.0/0"
  gateway_id             = aws_internet_gateway.igw2.id
}
```

Subnet Associations:

```
resource "aws_route_table_association" "assoc1" {
  subnet_id      = aws_subnet.subnet1.id
  route_table_id = aws_route_table.rt1.id
```

```
}

resource "aws_route_table_association" "assoc2" {
  subnet_id      = aws_subnet.subnet2.id
  route_table_id = aws_route_table.rt2.id
}
```

Step 7: Create Security Groups

security.tf

Security Group for VPC-1

```
resource "aws_security_group" "sg1" {
  name   = "vpc1-sg"
  vpc_id = aws_vpc.vpc1.id

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = -1
    to_port     = -1
    protocol    = "icmp"
    cidr_blocks = [var.vpc2_cidr]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Security Group for VPC-2

```
resource "aws_security_group" "sg2" {
  name   = "vpc2-sg"
  vpc_id = aws_vpc.vpc2.id

  ingress {
```

```
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = -1
    to_port     = -1
    protocol    = "icmp"
    cidr_blocks = [var.vpc1_cidr]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Step 8: Create VPC Peering

peering.tf

```
resource "aws_vpc_peering_connection" "peer" {
  vpc_id      = aws_vpc.vpc1.id
  peer_vpc_id = aws_vpc.vpc2.id
  auto_accept = true

  tags = {
    Name = "VPC-Peering"
  }
}
```

Step 9: Add Peering Routes

route.tf

```
resource "aws_route" "peer_route1" {
  route_table_id            = aws_route_table.rt1.id
  destination_cidr_block    = var.vpc2_cidr
  vpc_peering_connection_id = aws_vpc_peering_connection.peer.id
```

```
}

resource "aws_route" "peer_route2" {
  route_table_id            = aws_route_table.rt2.id
  destination_cidr_block    = var.vpc1_cidr
  vpc_peering_connection_id = aws_vpc_peering_connection.peer.id
}
```

Step 10: Launch EC2 Instances

ec2.tf

Get Latest Amazon Linux 2023 AMI

```
data "aws_ami" "amazon_linux" {
  most_recent = true

  owners = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*"]
  }
}
```

Server 1

```
resource "aws_instance" "server1" {
  ami                         = data.aws_ami.amazon_linux.id
  instance_type               = var.instance_type
  subnet_id                   = aws_subnet.subnet1.id
  vpc_security_group_ids      = [aws_security_group.sg1.id]
  key_name                    = var.key_name
  associate_public_ip_address = true

  tags = {
    Name = "Server-1"
  }
}
```

Server 2

```
resource "aws_instance" "server2" {
  ami                         = data.aws_ami.amazon_linux.id
  instance_type               = var.instance_type
  subnet_id                   = aws_subnet.subnet2.id
  vpc_security_group_ids      = [aws_security_group.sg2.id]
```

```
  key_name                    = var.key_name
  associate_public_ip_address = true

  tags = {
    Name = "Server-2"
  }
}
```

Step 11: Create Outputs

outputs.tf

```
output "server1_private_ip" {
  value = aws_instance.server1.private_ip
}

output "server2_private_ip" {
  value = aws_instance.server2.private_ip
}

output "vpc_peering_id" {
  value = aws_vpc_peering_connection.peer.id
}
```

Step 12: Terraform Execution

Format files:

```
terraform fmt
```

Initialize:

```
terraform init
```

Validate:

```
terraform validate
```

Review plan:

```
terraform plan
```

Deploy:

```
terraform apply
```

Approve:

```
yes
```

Step 13: Verify Resources

Check:

- VPCs
- Subnets
- Internet Gateways
- Route Tables
- Security Groups
- EC2 Instances
- VPC Peering Connection

Step 14: Test VPC Peering

SSH to Server-1:

```
ssh -i VPC-Prod-Terraform.pem ec2-user@<server1-public-ip>
```

Ping Server-2 Private IP:

```
ping <server2-private-ip>
```

SSH to Server-2:

```
ssh -i VPC-Prod-Terraform.pem ec2-user@<server2-public-ip>
```

Ping Server-1 Private IP:

```
ping <server1-private-ip>
```

Successful ping confirms:

- VPC Peering Working
- Route Tables Correct
- Security Groups Correct
- Private Communication Established

Useful Terraform Commands

```
terraform fmt
terraform init
terraform validate
terraform plan
terraform apply
terraform show
terraform state list
terraform destroy
```